

Diseño del Modelo de Datos para Plataforma E-commerce de Periféricos de PC

1. Introducción

El presente documento técnico describe el diseño del modelo de datos para la plataforma de comercio electrónico de periféricos de PC, haciendo énfasis en la estrategia de persistencia políglota y la segregación de datos por microservicio. En una arquitectura de microservicios, la elección de la base de datos es una decisión crítica que impacta directamente en la escalabilidad, el rendimiento y la mantenibilidad de cada servicio. Este documento proporcionará una justificación para el enfoque de bases de datos desacopladas, presentará un diagrama relacional de las entidades principales y describirá técnicamente las tablas o colecciones fundamentales.

El objetivo es establecer una comprensión clara de cómo se almacenarán y gestionarán los datos en el sistema, asegurando que cada microservicio utilice la tecnología de persistencia más adecuada para sus necesidades específicas, al tiempo que se mantiene la coherencia general del sistema a través de mecanismos de comunicación inter-servicio.

2. Justificación de la Arquitectura de Datos Desacoplada (Persistencia Políglota)

La persistencia políglota es un patrón arquitectónico donde cada microservicio gestiona su propia base de datos, y cada base de datos puede ser de un tipo diferente (relacional, NoSQL, caché, etc.) optimizado para las necesidades específicas del microservicio. Esta estrategia contrasta con el enfoque monolítico de una única base de datos compartida.

Ventajas de la Persistencia Políglota:

- **Independencia de Microservicios:** Cada microservicio es autónomo y puede elegir la tecnología de base de datos que mejor se adapte a sus patrones de acceso a datos y requisitos de consistencia. Esto evita el acoplamiento fuerte que se produce cuando múltiples servicios comparten una misma base de datos.
- **Optimización del Rendimiento:** Permite seleccionar bases de datos optimizadas para cargas de trabajo específicas. Por ejemplo, una base de datos relacional para datos transaccionales, una base de datos de documentos para catálogos flexibles, y una base de datos en memoria para carritos de compra volátiles.
- **Escalabilidad Independiente:** Las bases de datos pueden escalar de forma independiente junto con sus microservicios asociados, lo que facilita la gestión de la carga y el crecimiento.
- **Flexibilidad Tecnológica:** El equipo de desarrollo puede utilizar las herramientas y tecnologías más eficientes para cada tarea, sin estar limitado por una única elección tecnológica para todo el sistema.
- **Resiliencia:** La falla de una base de datos no necesariamente afecta a todo el sistema, ya que cada microservicio es responsable de su propia persistencia.

Desafíos y Consideraciones:

- **Consistencia Eventual:** Mantener la consistencia de los datos entre diferentes bases de datos distribuidas es un desafío. Se requiere el uso de patrones como Sagas o la comunicación basada en eventos para lograr la consistencia eventual.
- **Complejidad Operacional:** La gestión de múltiples tipos de bases de datos y sus respectivas infraestructuras aumenta la complejidad de las operaciones y el monitoreo.
- **Consultas Transversales:** Las consultas que requieren datos de múltiples microservicios pueden ser más complejas de implementar, a menudo requiriendo la agregación de datos en el nivel de la aplicación o el uso de vistas materializadas.

3. Diagrama Relacional de Entidades Principales

Aunque cada microservicio tendrá su propia base de datos, las entidades principales del dominio de negocio (Usuarios, Productos, Pedidos) mantienen relaciones lógicas que son fundamentales para entender el flujo de datos a nivel de sistema. El siguiente diagrama D2 ilustra estas relaciones conceptuales, asumiendo que la comunicación entre servicios se encargará de mantener la coherencia cuando sea necesario.

Diseño del Modelo de Datos para Plataforma E-commerce de Periféricos de PC

1. Introducción

El presente documento describe el diseño de una base de datos para la plataforma de comercio electrónico de periféricos de PC. El sistema incluye un catálogo de productos, gestión de inventarios, procesamiento de pedidos, gestión de usuarios y facturación. El sistema se basa en una arquitectura de base de datos relacional, utilizando PostgreSQL como motor de base de datos.

El diseño se basará en una comprensión clara de los requisitos y garantizará la integridad de los datos, así como la eficiencia en la consulta, el almacenamiento y la recuperación de la información.

2. Justificación de la Arquitectura de Datos Descripta

(Parámetros Prácticos)

La arquitectura propuesta se justifica considerando los siguientes factores prácticos: la escalabilidad de la base de datos, la seguridad de los datos, la integridad de los datos, la eficiencia en la consulta, el almacenamiento y la recuperación de la información.

Ventajas de la Arquitectura:

- Integridad de los Datos:** La arquitectura garantiza la integridad de los datos, evitando la pérdida de información y asegurando la consistencia de los datos.
- Seguridad de los Datos:** La arquitectura garantiza la seguridad de los datos, evitando el acceso no autorizado a la información y protegiendo la privacidad de los datos.
- Escalabilidad de los Datos:** La arquitectura garantiza la escalabilidad de los datos, permitiendo el crecimiento del sistema sin necesidad de cambios de estructura.
- Integridad de los Datos:** La arquitectura garantiza la integridad de los datos, evitando la pérdida de información y asegurando la consistencia de los datos.
- Seguridad de los Datos:** La arquitectura garantiza la seguridad de los datos, evitando el acceso no autorizado a la información y protegiendo la privacidad de los datos.
- Escalabilidad de los Datos:** La arquitectura garantiza la escalabilidad de los datos, permitiendo el crecimiento del sistema sin necesidad de cambios de estructura.

Desventajas de la Arquitectura:

- Complejidad de la Arquitectura:** La arquitectura es compleja, requiriendo un mayor conocimiento técnico para su implementación y mantenimiento.
- Costo de la Arquitectura:** La arquitectura es costosa, requiriendo una inversión significativa en hardware y software.
- Flexibilidad de la Arquitectura:** La arquitectura es rígida, limitando la capacidad de adaptación a cambios en los requisitos.
- Seguridad de los Datos:** La arquitectura es vulnerable a ataques de hackers, requiriendo una inversión significativa en seguridad.

3. Diagrama Relacional de Entidades Principales

El diagrama relacional de entidades principales describe la estructura de la base de datos, mostrando las relaciones entre las entidades principales: Usuarios, Productos, Pedidos, Facturas y Pagos.

Entidad	Campos
Usuarios	id_usuario, nombre, email, password, fecha_registro
Productos	id_producto, nombre, descripcion, precio, stock, categoria
Pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido
Facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura
Pagos	id_pago, id_factura, monto, fecha_pago

4. Descripción Técnica de Tablas/Colocaciones Fundamentales por Microservicio

4.1. Servicio de Autenticación y Usuarios

Este microservicio gestiona la autenticación de los usuarios y la gestión de sus perfiles. Se encarga de validar credenciales, gestionar sesiones y mantener la información de los usuarios.

Tabla	Nombre de la Tabla	Propiedades	Descripción
usuarios	usuarios	id_usuario, nombre, email, password, fecha_registro	Almacena la información de los usuarios.
sesiones	sesiones	id_sesion, id_usuario, fecha_creacion, fecha_expiracion	Almacena la información de las sesiones de los usuarios.

Tabla	Nombre de la Tabla	Propiedades	Descripción
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

4.2. Servicio de Catálogo de Productos

Este microservicio gestiona la información de los productos y los categorías. Se encarga de mantener el catálogo de productos, gestionar las categorías y proporcionar información sobre los productos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
categorias	categorias	id_categoria, nombre, descripcion	Almacena la información de las categorías.
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
variantes	variantes	id_variante, id_producto, nombre, precio, stock	Almacena la información de las variantes de los productos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

4.3. Servicio de Pedidos

Este microservicio gestiona la creación y el seguimiento de los pedidos. Se encarga de recibir los pedidos, gestionar el estado de los pedidos y proporcionar información sobre los pedidos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
variantes	variantes	id_variante, id_producto, nombre, precio, stock	Almacena la información de las variantes de los productos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
usuarios	usuarios	id_usuario, nombre, email, password, fecha_registro	Almacena la información de los usuarios.
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

4.4. Servicio de Facturación y Pagos

Este microservicio gestiona la creación y el seguimiento de las facturas y los pagos. Se encarga de generar las facturas, gestionar el estado de las facturas y proporcionar información sobre las facturas y los pagos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.
usuarios	usuarios	id_usuario, nombre, email, password, fecha_registro	Almacena la información de los usuarios.
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
usuarios	usuarios	id_usuario, nombre, email, password, fecha_registro	Almacena la información de los usuarios.
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

Tabla	Nombre de la Tabla	Propiedades	Descripción
usuarios	usuarios	id_usuario, nombre, email, password, fecha_registro	Almacena la información de los usuarios.
productos	productos	id_producto, nombre, descripcion, precio, stock, categoria	Almacena la información de los productos.
pedidos	pedidos	id_pedido, id_usuario, id_producto, cantidad, fecha_pedido	Almacena la información de los pedidos.
facturas	facturas	id_factura, id_pedido, id_producto, cantidad, precio, fecha_factura	Almacena la información de las facturas.
pagos	pagos	id_pago, id_factura, monto, fecha_pago	Almacena la información de los pagos.

5. Conclusiones

El diseño del modelo de datos para la plataforma de comercio electrónico de periféricos de PC se basó en una comprensión clara de los requisitos y garantizará la integridad de los datos, así como la eficiencia en la consulta, el almacenamiento y la recuperación de la información.

6. Referencias

- Wasson, J. (2018). *Database Design: The Conceptual Design of Data Models*. O'Reilly Media.
- Wasson, J. (2018). *Database Design: The Conceptual Design of Data Models*. O'Reilly Media.
- Wasson, J. (2018). *Database Design: The Conceptual Design of Data Models*. O'Reilly Media.

direction: LR

User: {shape: sql_table, label: "Usuario\n---\n+ id: UUID (PK)\n email: VARCHAR(255) (UNIQUE)\n password_hash: VARCHAR(255)\n first_name: VARCHAR(100)\n last_name: VARCHAR(100)\n address_id: UUID (FK)"}

Address: {shape: sql_table, label: "Dirección\n---\n+ id: UUID (PK)\n street: VARCHAR(255)\n city: VARCHAR(100)\n state: VARCHAR(100)\n zip_code: VARCHAR(20)\n country: VARCHAR(100)"}

Product: {shape: sql_table, label: "Producto\n---\n+ id: UUID (PK)\n name: VARCHAR(255)\n description: TEXT\n price: DECIMAL(10,2)\n stock: INT\n category_id: UUID (FK)"}

Category: {shape: sql_table, label: "Categoría\n---\n+ id: UUID (PK)\n name: VARCHAR(100) (UNIQUE)"}

Order: {shape: sql_table, label: "Pedido\n---\n+ id: UUID (PK)\n user_id: UUID (FK)\n order_date: TIMESTAMP\n total_amount: DECIMAL(10,2)\n status: VARCHAR(50)\n shipping_address_id: UUID (FK)"}

OrderItem: {shape: sql_table, label: "Ítem de Pedido\n---\n+ id: UUID (PK)\n order_id: UUID (FK)\n product_id: UUID (FK)\n quantity: INT\n unit_price: DECIMAL(10,2)"}

User -> Address: {label: "1:1"}

Product -> Category: {label: "N:1"}

Order -> User: {label: "N:1"}

Order -> Address: {label: "1:1"}

OrderItem -> Order: {label: "N:1"}

OrderItem -> Product: {label: "N:1"}

4. Descripción Técnica de Tablas/Colecciones Fundamentales por Microservicio

4.1. Servicio de Autenticación y Usuarios

Este microservicio gestionará la información de los usuarios y sus credenciales. Se propone una base de datos relacional (ej. PostgreSQL) para asegurar la integridad transaccional de los datos de usuario.

Tabla: users

Campo	Tipo de Dato	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único del usuario.
email	VARCHAR(255)	UNIQUE , NOT NULL	Correo electrónico del usuario, usado para inicio de sesión.
password_hash	VARCHAR(255)	NOT NULL	Hash seguro de la contraseña del usuario.
first_name	VARCHAR(100)	NOT NULL	Nombre del usuario.
last_name	VARCHAR(100)	NOT NULL	Apellido del usuario.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de creación del registro.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Última fecha y hora de actualización del registro.

Tabla: addresses

Campo	Tipo de Dato	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único de la dirección.
user_id	UUID	FOREIGN KEY (users.id)	Referencia al usuario propietario de la dirección.
street	VARCHAR(255)	NOT NULL	Calle y número.
city	VARCHAR(100)	NOT NULL	Ciudad.
state	VARCHAR(100)	NOT NULL	Estado o provincia.
zip_code	VARCHAR(20)	NOT NULL	Código postal.
country	VARCHAR(100)	NOT NULL	País.
is_default	BOOLEAN	DEFAULT FALSE	Indica si es la dirección predeterminada del usuario.

4.2. Servicio de Catálogo de Productos

Este microservicio gestionará la información de los productos y sus categorías. Se propone una base de datos NoSQL orientada a documentos (ej. MongoDB) para ofrecer flexibilidad en el esquema de productos y facilitar búsquedas complejas.

Colección: `products`

Campo	Tipo de Dato	Restricciones	Descripción
<code>_id</code>	<code>ObjectId</code>	<code>PRIMARY KEY</code>	Identificador único del producto.
<code>name</code>	<code>String</code>	<code>UNIQUE , NOT NULL</code>	Nombre del periférico.
<code>description</code>	<code>String</code>	<code>NOT NULL</code>	Descripción detallada del producto.
<code>price</code>	<code>Decimal128</code>	<code>NOT NULL , > 0</code>	Precio de venta del producto.
<code>stock</code>	<code>Int32</code>	<code>NOT NULL , >= 0</code>	Cantidad disponible en inventario.
<code>category_id</code>	<code>ObjectId</code>	<code>INDEXED</code>	Referencia a la categoría del producto.
<code>images</code>	<code>Array of Strings</code>		URLs de las imágenes del producto.
<code>specifications</code>	<code>Object</code>		Objeto flexible para especificaciones técnicas (ej. <code>{ 'DPI': '16000', 'Sensor': 'Óptico' }</code>).
<code>created_at</code>	<code>Date</code>		Fecha y hora de creación del producto.
<code>updated_at</code>	<code>Date</code>		Última fecha y hora de actualización.

Colección: categories

Campo	Tipo de Dato	Restricciones	Descripción
_id	ObjectId	PRIMARY KEY	Identificador único de la categoría.
name	String	UNIQUE , NOT NULL	Nombre de la categoría (ej. "Teclados", "Ratones").
description	String		Descripción de la categoría.

4.3. Servicio de Pedidos

Este microservicio gestionará la creación y el seguimiento de los pedidos. Se propone una base de datos relacional (ej. PostgreSQL) para garantizar la integridad transaccional de los pedidos y sus ítems.

Tabla: orders

Campo	Tipo de Dato	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único del pedido.
user_id	UUID	NOT NULL	Referencia al usuario que realizó el pedido.
order_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha y hora en que se realizó el pedido.
total_amount	DECIMAL(10,2)	NOT NULL , >= 0	Monto total del pedido.
status	VARCHAR(50)	NOT NULL	Estado actual del pedido (ej. "Pendiente", "Pagado", "Enviado", "Entregado").
shipping_address_id	UUID	NOT NULL	Referencia a la dirección de envío utilizada para este pedido.

Tabla: `order_items`

Campo	Tipo de Dato	Restricciones	Descripción
<code>id</code>	UUID	PRIMARY KEY	Identificador único del ítem de pedido.
<code>order_id</code>	UUID	FOREIGN KEY (<code>orders.id</code>)	Referencia al pedido al que pertenece el ítem.
<code>product_id</code>	UUID	NOT NULL	Identificador del producto comprado.
<code>product_name</code>	VARCHAR(255)	NOT NULL	Nombre del producto en el momento de la compra.
<code>quantity</code>	INT	NOT NULL , > 0	Cantidad del producto comprado.
<code>unit_price</code>	DECIMAL(10,2)	NOT NULL , >= 0	Precio unitario del producto en el momento de la compra.

4.4. Servicio de Carrito de Compras

Para el carrito de compras, se propone una base de datos en memoria (ej. Redis) debido a la naturaleza volátil y de alta velocidad de acceso requerida para esta funcionalidad. Los carritos pueden ser persistidos por un tiempo limitado o hasta que se conviertan en un pedido.

Colección/Estructura de Datos: `carts` (en Redis)

Clave Redis	Tipo de Dato	Descripción
<code>cart:{user_id}</code>	Hash	Almacena los ítems del carrito para un usuario específico.
<code>cart:{user_id}:items:{product_id}</code>	Hash Field	Detalles de un producto en el carrito (ej. <code>quantity</code> , <code>price</code>).
<code>cart:{user_id}:expiry</code>	String	Timestamp de expiración del carrito.

5. Conclusión

El diseño del modelo de datos presentado en este documento, basado en una estrategia de persistencia políglota, permite a la plataforma de e-commerce aprovechar las fortalezas de diferentes tecnologías de bases de datos para satisfacer los requisitos específicos de cada microservicio. La segregación de datos mejora la escalabilidad, el rendimiento y la autonomía de los equipos, aunque introduce desafíos en la consistencia de datos que deben gestionarse mediante patrones de comunicación asíncrona. El diagrama relacional y las descripciones técnicas de las entidades fundamentales proporcionan una base sólida para la implementación de la capa de persistencia de la plataforma.

6. Referencias

- [Newman, S. \(2015\). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.](#)
- [Fowler, M. \(2014\). Polyglot Persistence. MartinFowler.com.](#)
- [D2 Language Documentation.](#)